

LoadKaro — Full Monorepo Review

Date: 2 July 2026 · **Scope:** `LoadKaro/` (Expo SDK 54 mobile) + `admin-dashboard/` (Next.js 16) + shared Supabase backend · **Context:** MVP validation with the first 3,000–5,000 users in India, solo developer.

Every MUST-fix in this report was verified three ways: read in the source, checked against the **live** Supabase database (read-only queries: `pg_policies`, column grants, FK rules, security/performance advisors, row counts), and cross-checked by an independent reviewer. Findings that are fine for an MVP are called out as such on purpose — the "ignore for now" list matters as much as the fix list.

1. Executive summary

Is this MVP-ready? Not yet — but you are close. The architecture is genuinely solid for a solo build: RLS is enabled and correct on all 12 tables after your C1–C3 migration (verified live), the dashboard's service-role API guards every route before touching data, the auth bypass is fail-closed on production, secrets were never committed to git, and both apps build and type-check cleanly. What stands between you and launch is a short, concrete list — mostly small SQL/policy fixes and one cheap client change — not an architectural problem.

The 5 things that matter (all verified, all fixable in ~1–2 focused days):

1. **Any logged-in user can mark themselves "verified" (and upgrade their own subscription).** The `users` UPDATE policy only locks the `role` column; `verification_status`, `subscription_type`, `phone` and `unique_id` are self-editable via a direct API call. The verified badge is your core trust signal — this forges it. *(Finding S-1)* 2. **Truck verification is silently broken.** The upload succeeds but the final status flip to `pending` is rejected by RLS, so trucks never enter the review queue. Live data confirms it: 0 trucks are `pending`, and all 13 verification submissions are for users, none for trucks. *(Finding S-2)* 3. **A single crafted dashboard request can mass-delete or mass-mutate a whole table.** The `/api/admin/row` handler trusts a caller-supplied `idField`, so `DELETE {table:"loads", idField:"status", id:"open"}` deletes every open load using the service-role key (which bypasses RLS). *(Finding S-3)* 4. **A mistaken delete is unrecoverable.** Deletes are hard (no soft-delete), deleting one user cascades to all their loads/trucks/availabilities (verified FK rules), the audit log stores no snapshot of what was deleted, bulk-delete removes up to 100 rows behind a generic "Delete this record?" confirm — and the Free Supabase tier has **no backups**. *(Finding S-4 + Decision A)* 5. **Core flows can hang forever on a patchy connection.** There is no request timeout anywhere; a dropped 2G socket leaves the spinner spinning with no recovery but force-killing the app — for exactly the trucker-on-2G audience you're targeting. One-file fix. *(Finding M-1)*

Fix those five, enable Pro-tier backups, and you have a launchable MVP. Everything else is month-1 polish or explicitly deferrable.

Per-app one-liners:

- **Mobile app:** ~3 small fixes from ready (S-1, S-2 are DB policy one-liners; M-1 is one file). Auth, RLS, pagination, and code health are in good shape.
 - **Dashboard:** Safe, well-guarded foundation, but not ready for *your own* daily use until S-3 + delete-safety (S-4) are fixed and the **seeding workflow** is built or worked around (today it cannot create loads/trucks/availabilities at all).
-

2. Verification method & what's confirmed healthy

Confirmed against the live project (`ffbbujudebmtbuyssnsn`) and source on 2026-07-02:

- **Your C1–C3 "safe-launch QA migration" is live.** The old `users` "view users" `USING(true)` full-phone-exposure policy is **gone**, replaced by "view public listing contacts" (`auth.uid()=id` OR `user_has_public_listing(id)`); `auth.uid()` is wrapped as `(select auth.uid())` everywhere (C2); `location_states` + its read policy and the feed indexes exist (C3). The RLS backup files at the repo root are the *pre*-migration snapshot — do not treat them as current state.
- **RLS is enabled on all 12 public tables**; an event trigger auto-enables it on any new table.
- **Dashboard build:** `next build` succeeds cleanly (Next.js 16.2.2, 33 routes, TypeScript passes, zero warnings). * (The first build attempt failed on a Windows/OneDrive `.next` file lock — `EPERM rmdir` — not a code error; it passed after clearing `.next`. See §9.)*
- **Mobile type-check:** `tsc --noEmit` passes with no errors.
- **Secrets in git history: clean.** No real key was ever committed to any of the three repos — only `.env.example` templates. The service-role key lives only in the git-ignored `admin-dashboard/.env` and is correctly server-only (never `NEXT_PUBLIC_`, never inlined into the client bundle by `next.config.ts`).
- **Auth flow correctness:** the DEF_001 OTP-race fix is present and working; role self-selection at registration is server-enforced (a user cannot register as admin/moderator); the "auth user exists but profile insert failed" edge case self-heals on next launch.
- **MSG91 readiness:** the app's only auth surface is `signInWithOtp` / `verifyOtp({type:'sms'})` — fully provider-agnostic. **Zero app-side changes are needed** for the Twilio→MSG91 swap; it lives entirely in the Supabase Send-SMS auth hook. (One caveat: fix the error-mapping bug M-9 first so you can actually see delivery failures during cutover.)

3. Findings — Bucket 1: MUST FIX before public launch

Each finding: app · file · why it matters at your scale · smallest fix.

S-1 · Users can self-grant "verified" status and upgrade their own subscription — SECURITY

App: DB / mobile · **Where:** `users` RLS policy "update own user" ; exploitable via the anon-key client any authenticated user holds. **Verified:** Live column grants show `authenticated` has UPDATE on `verification_status` , `subscription_type` , `phone` , `unique_id` , `role` , `name` . The policy's `WITH CHECK` only constrains `role` (it must equal the stored role). So a user with a valid OTP session can `PATCH /rest/v1/users?id=eq.<self>` with `{"verification_status":"verified"}` or `{"subscription_type":<premium>}` and it succeeds.

Why it matters: The verified badge is shown to counterparties on every load and truck card — it is the platform's entire trust model. Anyone can forge it in one request; premium is presumably paid. **Smallest fix:** Tighten the `WITH CHECK` to snapshot-lock the sensitive columns (same pattern the role-lock already uses), while still permitting the one legitimate self-transition the app needs (`verification_status` → 'pending' on document submit):

```

DROP POLICY "update own user" ON public.users;
CREATE POLICY "update own user" ON public.users FOR UPDATE TO authenticated
USING ((SELECT auth.uid()) = id)
WITH CHECK (
  (SELECT auth.uid()) = id
  AND role IS NOT DISTINCT FROM (SELECT u.role FROM public.users u WHERE u.id = (SEL
  AND subscription_type IS NOT DISTINCT FROM (SELECT u.subscription_type FROM public.users u WHERE u.id = (SEL
  AND phone IS NOT DISTINCT FROM (SELECT u.phone FROM public.users u WHERE u.id = (SEL
  AND unique_id IS NOT DISTINCT FROM (SELECT u.unique_id FROM public.users u WHERE u.id = (SEL
  AND ( verification_status IS NOT DISTINCT FROM (SELECT u.verification_status FROM public.users u WHERE u.id
      OR verification_status = 'pending':user_verification_status_enum )
);

```

S-2 · Truck verification is silently broken (RLS rejects the status flip) — BROKEN FEATURE

App: DB / mobile · **Where:** `trucks` policy `"update trucks"` (WITH CHECK) vs `LoadKaro/lib/verificationUpload.js:160-164`. **Verified:** The upload flow sets `trucks.verification_status = 'pending'`, but the owner branch of the trucks UPDATE policy requires the row to remain `unverified`, so the write is rejected (42501) and the user sees "Documents uploaded but status update failed." Live data confirms this has never succeeded: **0 pending trucks**, and all 13 `verification_submissions` are `entity_type='user'`, none `truck`. (User verification works today only because S-1 leaves the users policy too loose — fixing S-1 makes the `→pending` exception load-bearing for both.) **Why it matters:** Truck owners are half your marketplace; their trucks can never get verified, so the trust badge never appears on truck listings. **Smallest fix:** Allow the `unverified → pending` transition in the owner branch:

```

DROP POLICY "update trucks" ON public.trucks;
CREATE POLICY "update trucks" ON public.trucks FOR UPDATE TO authenticated
USING (
  ((owner_id = (SELECT auth.uid())) AND (verification_status = 'unverified':truck_verification_status_enum))
  OR (EXISTS (SELECT 1 FROM users WHERE users.id = (SELECT auth.uid()) AND users.role = 'admin':user_role_enum)
)
WITH CHECK (
  ((owner_id = (SELECT auth.uid())) AND (verification_status IN ('unverified','pending')))
  OR (EXISTS (SELECT 1 FROM users WHERE users.id = (SELECT auth.uid()) AND users.role = 'admin':user_role_enum)
);

```

S-3 · One request can mass-delete / mass-mutate a table via caller-controlled `idField` — SECURITY

App: dashboard · **Where:** `admin-dashboard/src/app/api/admin/row/route.ts:81, 113` (PATCH) and `:243, 258` (DELETE). **Verified in code:** `idField = body.idField ?? "id"` is used unvalidated as `.update(...).eq(idField, id) / .delete().eq(idField, id)` with the **service-role** client (bypasses RLS). Examples: admin `DELETE {table:"loads", idField:"status", id:"open"}` → deletes every open load; moderator `PATCH {table:"users", idField:"role", id:"shipper", patch:{verification_status:"rejected"}}` → rejects every shipper at once. **Why it matters:** The realistic threat is a compromised staff session or a client bug that amplifies one action into a table-wide one (browser CSRF is hard here, so this isn't a drive-by). Given hard deletes + no backups on Free, the blast radius is catastrophic. Cheap to close. **Smallest fix:** Validate `idField` against a tiny per-request allowlist in both handlers:

```
const ALLOWED_ID_FIELDS = new Set(["id", "unique_id"]);
if (!ALLOWED_ID_FIELDS.has(idField)) {
  return NextResponse.json({ error: "Invalid idField" }, { status: 400 });
}
```

S-4 · A mistaken delete is permanent and unrecoverable — DATA INTEGRITY

App: dashboard + DB + ops · **Where:** `common-data-table.tsx` delete/bulk-delete → `row/route.ts:223-276`; live FK rules; Supabase tier. **Verified live:** Deleting a **user** cascades (`ON DELETE CASCADE`) to their `loads` (`posted_by`), `trucks` (`owner_id`), and `availabilities` (`owner_id`); deleting a truck cascades to its availabilities; deleting a load cascades to `admin_alerts`. Deletes are hard (no `deleted_at` anywhere). The audit log writes `details: {}` for deletes — so even the log can't reconstruct what was destroyed. Bulk-delete removes up to 100 selected rows in a non-transactional loop behind a generic `window.confirm("Delete N selected record(s)?)`. On the Free tier there are **no backups** (see Decision A). **Why it matters:** "Select all → Delete selected → OK" on the Loads or Users page permanently erases marketplace data — and one user-delete silently takes their whole footprint with it. For a solo operator this is the single most likely way to lose data catastrophically. **Smallest fix (do all four, each tiny):** 1. Enable **Supabase Pro** for daily backups (Decision A) — this is the real safety net. 2. In the DELETE handler, `select("*")` the target row(s) first and store the snapshot in the audit `details`, so any delete is manually restorable from the activity log. 3. Replace `window.confirm` with the shadcn `AlertDialog` that names the entity's `unique_id`, and for bulk-delete require typing the row count. 4. When `table === "users"`, also call `supabase.auth.admin.deleteUser(id)` so a deleted user's login doesn't linger (today the data is wiped but the auth account survives).

M-1 · No request timeout — core flows hang forever on patchy 2G — UX (audience-critical)

App: mobile · **Where:** `LoadKaro/lib/supabase.js:23-34` (default fetch, no timeout); affects every list load and every submit. **Verified:** No `AbortController` /timeout/retry anywhere in the app. React Native `fetch` has no default read timeout, so a stalled socket (signal present, data dead — the common 2G failure) leaves `loading / saving` true indefinitely; several bootstrap fetches also lack `try/finally` so a *rejected* promise never resets the spinner. **Why it matters:** Your stated core users are truckers on patchy 2G. A "Post a Load" or OTP screen frozen with no way out is a direct abandon, and it's the most common real-world condition they'll hit. **Smallest fix:** Give the Supabase client a timeout-wrapped fetch — one change covers PostgREST, auth, and storage:

```
createClient(url, key, {
  global: { fetch: (u, o) => fetch(u, { ...o, signal: AbortSignal.timeout(30000) }) },
  auth: { /* existing */ },
});
```

Then wrap the four bootstrap fetches (`UploadLoadScreen`, `ShipperPostLoadScreen`, `EditTruckScreen`, `AddTruckScreen`) in `try { ... } finally { setLoading(false); }`.

M-2 · Seeding workflow can't run from the dashboard — BLOCKS YOUR STATED LAUNCH PLAN

App: dashboard · **Where:** `row/route.ts:134` (`INSERT_ALLOWED_TABLES = {locations}` only); bulk-actions bar has only "Delete selected". **Verified:** The dashboard has **no way to create** loads, trucks, or availabilities — POST is hard-restricted to `locations`. And there's no bulk-close, so "seed then close them down" is N individual clicks while the only bulk tool is the destructive one. This directly blocks the seeding tool you described. **Smallest fix (pick the interim if short on time):**

- ***Interim (0 dashboard code):*** seed via the mobile app using two or three dedicated seed accounts, then add just the **"Close selected"** bulk button (~15 lines; server already allows `status:'closed'` for both roles and audits it as `close_record`).
- ***Proper:*** add `loads / availabilities` to `INSERT_ALLOWED_TABLES` with a minimal validated create form (mirror the existing `AddPlaceForm` in `admin/locations/page.tsx`), plus the `is_seed` flag below.
- **Seed tagging (do this regardless):** add one boolean per content table so analytics can exclude seeds and cleanup is one query:

```
`` sql ALTER TABLE public.loads ADD COLUMN is_seed boolean NOT NULL DEFAULT false; ALTER TABLE
public.availabilities ADD COLUMN is_seed boolean NOT NULL DEFAULT false; ALTER TABLE public.trucks ADD
COLUMN is_seed boolean NOT NULL DEFAULT false; ` Set is_seed=true only on the seed insert path (service-
role/dashboard). Cleanup: DELETE FROM loads WHERE is_seed; (children before parents). Analytics:
WHERE NOT is_seed`. Note: a seed listing's "Call" button surfaces the owner's phone, so seed under accounts
whose phone is your ops/support number.
```

4. Findings — Bucket 2: SHOULD FIX in the first month

Grouped by area. All are real but safe to ship without, for a few weeks at your scale.

Mobile — marketplace correctness

- **M-3 · FindReturnLoad pagination skips loads.** `screens/FindReturnLoadScreen.js:129-133` pages on a non-unique `loading_date` DATE with strict `.gt()`, so when ≥ 20 loads share a date the rest become unreachable on the return-load screen. **Promote to MUST if you bulk-seed loads with the same `loading_date`** (likely at launch). Fix: page by `created_at` like the other screens, or add an `id` tiebreaker.
- **M-4 · "Clear Filter"/re-Apply blanks the list.** `ViewLoadsScreen.js:244-254` (and the two sibling browse screens) call `setData([])` but only refetch when the filter IDs actually change, so tapping Clear with no active filter empties a seeded-to-look-alive marketplace. Fix: call `fetchPage(null,false)` directly on apply/clear.
- **M-5 · Verification upload has no progress UI and orphans partial state.** `ManageTrucksScreen.js:140` closes the modal before the multi-minute upload; `verificationUpload.js` uploads sequentially with no timeout, leaving orphan submission rows/objects on 2G failure. Fix: keep a blocking "Uploading 1/3..." state; insert each doc row right after its upload.
- **M-6 · Bulk-seeded rows share `created_at`, breaking the `created_at` cursor.** A single `INSERT...SELECT` gives identical timestamps; rows at a page boundary get skipped. Fix: stagger seed `created_at` by seconds, or add an `id` tiebreaker to the cursor.
- **M-7 · Shippers can't close their own loads; `matched` renders wrong.** No mobile UPDATE on `loads` exists, so loads stay `open` until an admin closes them; and a `matched` load (only the admin can set it) shows in the shipper's own list as a raw untranslated pill / drops out of the "active" count (`ShipperMyLoadsScreen.js:33` , `ShipperHomeScreen.js:72,252`). Fix: add a "Mark closed" button (mirror `ManageAvailabilitiesScreen`) and map `matched` to a positive pill + add the `load_status_matched` i18n key (4 languages) — or drop `matched` from the dashboard options until mobile supports it.
- **M-8 · tel: calls have no failure handling.** Four call sites `Linking.openURL('tel:...')` un-awaited; on a device without a dialer the primary action silently no-ops. Fix: `.catch(() => Alert.alert(t('unavailable'), phone))`.

Mobile — auth / OTP

- ****M-9 · SMS *send* failures are mislabeled "Wrong OTP."** `utils/authErrors.ts:15-17` **regex-matches send-side errors as invalid-OTP**. Fix this before the MSG91 cutover** or you won't be able to tell real delivery failures apart from user typos.

- **M-10 · No "Resend OTP" button / no verify timeout.** India SMS is routinely delayed; with no resend and no cooldown UI, a user who doesn't get the code must back out and restart. (M-1 covers the timeout half.) Fix: add a resend button with a 30–60s countdown.
- **M-11 · Offline sign-out and offline cold-start log the user out incorrectly.** `authStore.ts:64-70` treats any profile-fetch failure as "no account" (logs out a valid user on a flaky cold start); `signOut()` throws before clearing local state offline. Fix: only sign out on Postgres `PGRST116` (no-rows); use `signOut({scope:'local'})` and clear state in `finally`.
- **M-12 · Post-verify errors are hardcoded English.** `syncUserAfterOtp.js` throws English strings shown raw on the OTP screen, though translations exist. Fix: throw error `*keys*` and map through `t()`.

Dashboard — safety & workflow

- **M-13 · Verification review: approve is one-click with no confirm, reject reason is optional and never shown to the user, and the decision write is non-atomic / last-write-wins.** `verification-docs-modal.tsx` + `decision/route.ts`: approve works even with zero documents; two moderators can both decide (no `.eq("status", "submitted")` claim guard); `rejection_reason` is written but `MyDocumentsModal.js:182` never renders it. Fix: require a reject reason, claim the submission first (`.eq("status", "submitted").select()` → 409 if empty) before updating the entity, bind `submission_id` to `entity_id`, and surface `rejection_reason` on mobile.
- **M-14 · Blur-saving enum/boolean cells are a misclick waiting to happen.** `common-data-table.tsx:1376-1407` opens dropdowns on single click and commits on blur — including `role` and `verification_status`. Fix: confirm dialog on `role / verification_status / status` changes.
- **M-15 · `moderator/create` can silently demote an existing admin (sole-admin lockout).** `moderator/create/route.ts:86-95` unconditionally sets an existing matched user's role to `moderator`. Fix: refuse if `existing.role` is already `admin / moderator`.
- **M-16 · Dead browser-client write fallback re-opens the RLS bypass if anyone forgets a prop.** `useServiceRoleApi` defaults to `false`; every live page sets it, but the default-false fallback (and the unused `shippers-table.tsx`) bypasses the allowlist + audit. Fix: delete the non-API branches and the prop; delete `shippers-table.tsx`.
- **M-17 · Dashboard `.or()` search injection + `select` embed traversal.** `search/route.ts` interpolates `q` raw into `.or()`; `data/route.ts` allows PostgREST embed syntax so a moderator can traverse FKs into non-allowlisted tables (and read `audit_log / id_sequences`). Staff-only, low impact, but close it: strip PostgREST metacharacters from `q`; restrict `select` to a per-table column set; add `audit_log / id_sequences` to `ADMIN_ONLY_TABLES`.
- **M-18 · `verification/signed-url` and `verification/decision` don't scope to the submission.** Any object path in the bucket can be signed; any entity id can be marked verified. Fix: confirm `path` belongs to the submission's documents; bind entity↔submission.

DB hardening

- **M-19 · `phone_exists` RPC is an anon-callable phone-number oracle.** Advisor-flagged; needed pre-OTP by the registration check, so keep it callable but add Supabase API rate-limiting / captcha on the auth flow, or move the check inside the Send-SMS edge function. Don't ignore past month 1.
- **M-20 · `generate_unique_id` / `auto_expire_listings` / `luhn_check_digit` are `authenticated -executable`.** Any logged-in user can burn ID sequences or trigger the expiry job via REST. Fix: `REVOKE EXECUTE ... FROM anon, authenticated;` (trigger functions still fire regardless).
- **M-21 · `EditTruck` shows success on a 0-row update.** `EditTruckScreen.js:134` updates by id with no `.select()`; an RLS-filtered no-match returns success. Fix: append `.eq('verification_status', 'unverified').select('id')` and treat empty as an error.
- **M-22 · Rotate the `.mcp.json` Supabase personal-access token.** It's git-ignored (never committed) but a real `sbp_...` token sits in the working tree; rotate it and, if possible, source it from an env var. Also move the

untracked `rls_backup_2026-07-01.sql` out of the OneDrive-synced repo root.

Observability & KPIs — install the minimal stack in Decision B and wire the KPI capture in Decision C during month 1.

5. Findings — Bucket 3: EXPLICITLY IGNORE for MVP (with reasons)

These were checked and are **fine to ship as-is** — do not spend time here now.

- `EXPO_PUBLIC_TEST_MODE` / `EXPO_PUBLIC_TEST_OTP` — confirmed dead: read by zero source files. *Delete the 4 `eas.json` blocks and 2 `.env` lines* for tidiness (it looks like a backdoor toggle but isn't), otherwise harmless.
 - **Supabase "leaked password protection" advisor** — irrelevant; auth is phone-OTP, there are no passwords.
 - **Unindexed FKs (3) and unused indexes (13), incl. the new feed indexes** — all INFO-level; "unused" only because the DB is near-empty (30 users, 14 loads). Keep them; they'll serve the feeds at scale. No action.
 - **Fetch-all on `ManageTrucks` / `ManageAvailabilities`** — scoped to one owner's own rows; trivial volume. Fine, revisit only past a few hundred rows per owner.
 - `created_at` -cursor **microsecond collisions in organic traffic** — effectively nil; the only real risk is bulk seeding (M-6). The keyset design is otherwise better than offset paging.
 - `active_locations` **view has broad anon grants** — verified harmless: it's a JOIN view (not auto-updatable) and the base tables have no write policies, so writes fail regardless. Optional one-line REVOKE.
 - **Admin/moderator get an empty dashboard on mobile** — `BUTTON_CONFIG` is empty for them by design; they use the web dashboard. Not a dead end (logout works).
 - **Committed `rls_backup_2026-05-25.sql`** — it's policy *structure*, not credentials; no secret exposure. (`*.sql` isn't gitignored — add it if you like.)
 - **~1,300 lines of dead code** (`PaginationBar.js` , `config/listScreenLabels.ts` , deprecated `verificationUi` aliases, empty `src/` , `shippers-table.tsx` , `types/users.ts` , orphaned `interests` / `show_interest` i18n keys) — safe to delete in one cleanup commit, but it costs you nothing at runtime. Not blocking.
 - **Auth-listener not unsubscribed / redundant `getUser()` / 400ms refetch on auth events** — benign; the listener lives for the app's lifetime and never double-inserts. Fine.
 - **Minor auth polish** — 15s stuck-session net is slightly aggressive on 2G (bump to ~30s later); double-tap Verify is harmless (OTP already consumed). Fine.
-

6. RLS & data-access matrix (verified live, 2026-07-02)

Mobile ships the anon key, so **every** mobile table access is governed by these policies. Verdicts reflect the *live* post-C1–C3 state.

Table / object	Mobile operations	Live policy (summary)	Verdict	
users	SELECT own + counterparties' contact; INSERT self; UPDATE own name (+ →pending)	SELECT: own OR user_has_public_listing(id) ; INSERT: own id + role∈{shipper,truck_owner,broker}; UPDATE: own id, only role locked	⚠ S-1 (self-escalation). SELECT/INSERT correct.	
loads	INSERT own; SELECT own + market (open)	INSERT posted_by=uid +role shipper; SELECT own OR truck_owner/broker/admin/mod; UPDATE/DELETE poster or admin	✅ OK	
trucks	INSERT own; SELECT own + listed; UPDATE own; →pending ; DELETE own	UPDATE (owner AND unverified) OR admin — blocks →pending	⚠ S-2 (breaks verification). Reads/insert/delete OK.	
availabilities	INSERT own; SELECT own + market (available); UPDATE own status	INSERT owner+role; SELECT owner or shipper/broker/admin/mod; UPDATE/DELETE owner or admin	✅ OK	
truck_variants	SELECT active	USING(true) , no writes	✅ OK	
locations / active_locations	mobile reads the active_locations view	view is security_invoker=true over locations + location_states , both USING(true) read	✅ OK — dropdowns work; the "missing policy" was a false alarm (it's a view).	
location_states	via view only	read location_states USING(true)	✅ OK	
verification_submissions	INSERT own; SELECT own	INSERT submitted_by=uid + entity ownership; SELECT own/owned-truck; no UPDATE/DELETE	✅ OK (user can't self-approve)	
verification_documents	INSERT/SELECT own submission	gated via parent submission	✅ OK	
storage: verification-docs	upload {user\	truck}/{id}/...; signed-URL read	3 folder-scoped policies (own-user / owned-truck)	✅ OK — no cross-user read; upsert:false prevents overwrite
admin_alerts / audit_log / id_sequences	not touched by mobile	admin/mod or service-role only	✅ OK — anon fully locked out	

Phone-harvesting verdict: closed for a 3–5k-user MVP. Post-C1, a user row is readable only if that user currently has an **open** load or **available** availability (or it's your own row) — which is exactly what the call buttons need and no more; the whole-table dump is gone. Two residuals, both month-1: the policy exposes the whole row (so **subscription_type** / **unique_id** leak alongside phone — scope a contacts view later), and **phone_exists** still allows single-number testing (M-19).

7. Per-app readiness verdicts

Mobile app — Close. ~3 fixes from ready.

Auth is well-built (DEF_001 fix verified, role escalation server-blocked, partial-failure self-heals), RLS is correct post-migration, pagination is proper keyset on the main browse screens, i18n covers 4 languages, and the code is unusually clean for a solo MVP (`tsc` passes, one stray `console.log`, no TODOs). **Blockers:** S-1 and S-2 (both one-line policy fixes) and M-1 (one-file timeout). After those, the mobile app is launchable; M-3/M-4/M-9 should follow within days because they touch launch-seeding and the MSG91 cutover.

Dashboard — Safe foundation; not yet ready for your own daily use.

The security architecture is genuinely good: every `/api/admin/*` route calls `requireDashboardAccess()` before touching the service-role client, the moderator/admin split is enforced server-side (patch-allowlist + 403s, not just hidden UI), the auth bypass is fail-closed on Vercel, and the service-role key is server-only and uncommitted. It builds clean on Next 16. **Blockers for relying on it:** S-3 (mass-mutate via `idField`) and S-4 (unrecoverable deletes), plus M-2 (it can't actually create the listings you want to seed). The bar here is "safe + functional for you + one moderator," and it clears that on **safety** once S-3/S-4 land; on **functionality** it needs the seeding path.

8. Documentation drift (appendix — verify code, not docs, until refreshed)

The four `DOCS_*.md` files are excellent but are April-2026 snapshots that missed three change waves. Trustworthy for auth flows, the verification pipeline, and API endpoint shapes; **dangerously stale** for the connection model, the database, and dashboard permissions.

- **Headline drift:** the entire **"Show Interest" / interests / leads system** is documented across all six docs (architecture §16, README, mobile & dashboard docs, DB doc §2.11, a dedicated diagram) but the `interests` table **does not exist in the live DB** and there is **zero code** for it in either app — connection is a direct `tel:` call. A future contributor would build against a table that isn't there. **Rewrite these sections to "direct call; interests/leads was removed."**
- **RLS section is two migrations behind** (doc says 31+3 policies incl. `interests` and a 5th `loads` policy; live has 28 public policies, plus the C1–C3 rewrite). It's your only written security reference — regenerate it from the live schema.
- **Undocumented since April:** the June shipper bottom-tab redesign (docs still say "stack only, no tabs"), the Locations/coverage admin area, Luhn display IDs (`id_sequences`), `POST /api/admin/row`, and moderator detail pages (docs say those are admin-only). File/screen inventories are materially off (e.g. `FindReturnLoadScreen`, the whole `screens/shipper/` tree, `AnimatedSplash`, `ErrorBoundary` are undocumented).

9. Build, type-check & Play-review notes

- **Dashboard build:** clean (`next build`, Next 16.2.2, 33 routes, TS passes, no warnings). Admin pages prerender as static shells — fine, since data comes from the self-guarded API and middleware gates the routes at request time.
- **Mobile type-check:** `tsc --noEmit` clean.
- **Windows/OneDrive caveat:** the repo lives under OneDrive; the first build failed on `EPERM rmdir '.next'` (a sync/file-lock artifact, not code). Building on a non-synced path (or pausing OneDrive) avoids it — worth doing for your own sanity during development.

- **Play review (mobile):** `app.json` has no `android.permissions` array; Expo prebuild will add `INTERNET`, `ACCESS_NETWORK_STATE` (from netinfo), and template defaults incl. `SYSTEM_ALERT_WINDOW` and legacy storage perms. None are "dangerous"/runtime perms, but `SYSTEM_ALERT_WINDOW` + storage perms show up in the Play listing and invite reviewer questions. One-line hardening:

```
``json "android": { "blockedPermissions": [ "android.permission.SYSTEM_ALERT_WINDOW",  
"android.permission.READ_EXTERNAL_STORAGE", "android.permission.WRITE_EXTERNAL_STORAGE" ]} ``
```

- **EAS env fragility:** builds get Supabase vars only because `.easignore` deliberately ships your local `.env`; a build from another machine would silently produce an unconfigured app. Fix: `eas env:create` the two `EXPO_PUBLIC_SUPABASE_*` vars on the project.
- **Vercel deploy:** no blockers. Set `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY` (or the `EXPO_PUBLIC_*` equivalents — `next.config.ts` maps them), and `SUPABASE_SERVICE_ROLE_KEY` in Vercel env. `allowedDevOrigins` is dev-only; bypass is fail-closed even if the flag were copied.

10. Decision A — Supabase tier

Recommendation: Pro, \$25/month (one project, both apps). Upgrade before public launch.

The deciding factor is **not** capacity — the Free tier's 50,000 MAU and 500 MB DB comfortably cover 3–5k users. It's **backups and pausing**:

	Free	Pro (\$25/mo)
MAU	50,000	100,000
DB size	500 MB	8 GB included
Backups	none	daily, 7-day retention
Auto-pause	after 7 days idle	never
Egress	5 GB	250 GB
Support	community	email

With hard deletes, cascading FKs, and no soft-delete (S-4), running the marketplace on a tier with **no backups** is the single biggest data-loss risk in this whole review. Pro's daily backups are the safety net that makes S-4 survivable. Auto-pause alone also disqualifies Free for anything user-facing. **Skip** Point-in-Time Recovery (a ~\$100/mo add-on) for now — daily backups are enough for an MVP. Budget \$25/mo, with a \$10–20 buffer once real traffic starts (watch the egress/DB lines).

*Sources: [Supabase Pricing](#) · [UI Bakery breakdown](#) · [MetaCTO cost guide](#)

11. Decision B — Minimal observability (one-person team, both apps)

Install exactly this; skip everything else.

Need	Tool	Plan	Setup
RN crash reporting	<code>@sentry/react-native</code> (Expo plugin)	Free Developer (5k errors/mo, 1 user)	Add the Expo plugin + DSN; upload source maps via the EAS hook.
Next.js API/route errors	<code>@sentry/nextjs</code>	same Sentry org (shared 5k/mo)	<code>withSentryConfig</code> + DSN; captures API-route exceptions with stack traces.
DB / auth logs	Supabase built-in Logs & Reports	included	No install. Check weekly; run <code>get_advisors</code> after any schema change.
Uptime	UptimeRobot (or Better Uptime) free	Free	One monitor on the dashboard URL + one on a Supabase REST health path; email/SMS on down.
OTP delivery rate (post-MSG91)	MSG91 delivery dashboard + Supabase auth logs	included	Watch delivery % during/after cutover; fix M-9 first so send failures are visible.

Alerts that matter: Sentry "new issue" + error-spike email; uptime-down alert. **Skip for MVP:** performance/APM, session replay, tracing, anything Datadog-class. One Sentry account covering both apps gives you a single place to see "is anything on fire," which is all a solo operator needs.

*Sources: [Sentry Pricing](#) · [Sentry pricing 2026 breakdown](#)

12. Decision C — KPIs for idea validation

Prefer capturing these as **SQL views/queries surfaced on the admin dashboard** (which already has the data table + service-role access). The one metric that needs new instrumentation is call/contact tracking — flagged below.

#	KPI	Definition	Simplest capture with your stack
1	Signup → OTP success	verified signups ÷ OTP send attempts	Supabase auth logs (send vs verify events); approximate day-1 via <code>users.created_at</code> count.
2	Activation	% of new users who post their first load / availability within 48h	SQL: join <code>users.created_at</code> to first <code>loads / availabilities</code> by <code>posted_by / owner_id</code> .
3	Listing → contact rate ⚠	calls initiated per active listing	Needs instrumentation — the <code>tel:</code> tap isn't logged today. Add a lightweight <code>contact_events</code> insert (or Sentry/analytics event) on call-button tap. Highest-value new metric.
4	Supply-demand liquidity	ratio of <code>open</code> loads to <code>available</code> trucks, by lane/state	SQL on <code>loads / availabilities</code> grouped by origin/destination location.
5	D7 / D30 retention	% of a signup cohort active again at day 7 / 30	<code>auth.users.last_sign_in_at</code> vs <code>created_at</code> cohorts (coarse but free); precise later via login events.
6	Seeded vs real engagement	listings & contacts on <code>is_seed=true</code> vs real rows	SQL filtered on the <code>is_seed</code> flag (M-2). Lets you prove real demand exists independent of seeds.
7	Verification completion	% of users/trucks reaching <code>verified</code>	SQL: <code>verification_status</code> distribution on <code>users / trucks</code> (fix S-2 first so trucks can complete).
8	Active-lane concentration	top origin→destination pairs by listing volume	SQL group-by on location FKs — tells you where to focus seeding/marketing.

13. One-page pre-launch checklist

MUST — before public launch

- **S-1** Replace the `users` "update own user" policy (lock `verification_status / subscription_type / phone / unique_id`; allow `→pending`).
- **S-2** Replace the `trucks` "update trucks" policy to allow `unverified → pending`; confirm a truck can reach `pending` then `verified` end-to-end.
- **S-3** Validate `idField` against `{id, unique_id}` in `row/route.ts` PATCH **and** DELETE.
- **S-4** Enable Pro backups; snapshot deleted rows into the audit log; typed-count confirm dialog on delete/bulk-delete; delete the auth user on user-delete.
- **M-1** Add a 30s timeout to the Supabase client fetch; wrap the 4 bootstrap fetches in `try/finally`.
- **M-2** Ship the seeding path: `is_seed` columns + "Close selected" bulk button (+ create form, or seed via mobile accounts interim).
- **A** Upgrade Supabase to **Pro (\$25/mo)**.

SHOULD — first month

- M-3/M-4/M-6 pagination + filter + seed-timestamp fixes · [] M-5 upload progress · [] M-7 close-load + `matched` handling · [] M-8 `tel:` catch
- M-9 fix SMS-send error mapping (**before MSG91 cutover**) · [] M-10 resend button · [] M-11 offline sign-out/cold-start · [] M-12 i18n post-verify errors
- M-13 verification confirm + required reason + atomic/claim-first + show reason on mobile · [] M-14 confirm on role/status edits · [] M-15 sole-admin guard · [] M-16 remove browser-write fallback · [] M-17/M-18 API hardening
- M-19 rate-limit `phone_exists` · [] M-20 REVOKE execute on maintenance functions · [] M-21 EditTruck 0-row check · [] M-22 rotate `.mcp.json` token
- **B** Install Sentry (RN + Next), 1 uptime monitor · [] **C** Wire KPI queries into the dashboard + add `contact_events`

IGNORE for MVP (noted so you don't second-guess): delete `EXPO_PUBLIC_TEST_MODE / TEST_OTP` (cosmetic), leaked-password advisor, unused indexes, dead code cleanup, docs refresh (do opportunistically — but treat code as truth until then).

Housekeeping before touching any of the above: the QA-pass changes in both apps and the C1–C3 migration are currently **uncommitted** in the working tree — commit them first so this work has a clean baseline.